

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Факультет информационных технологий и прикладной математики
Кафедра вычислительной математики и программирования

**Лабораторная работа №2 по курсу
«Операционные системы»**

**Тема работы
“Потоки”**

Студент: Салихов Руслан Ринатович
Группа: М8О-203Б-23
Вариант: 6

Преподаватель: Миронов Евгений Сергеевич
Оценка: _____
Дата: _____
Подпись: _____

Москва, 2024

Постановка задачи

Разработать программу на языке C, выполняющую обработку данных в многопоточном режиме. Использовать стандартные средства создания потоков операционной системы (Windows/Unix). Необходимо предусмотреть ограничение максимального количества потоков, работающих одновременно, которое задается параметром при запуске программы. Вариант 6. Произвести перемножение двух матриц, содержащих комплексные числа. Реализовать стратегию, адаптирующуюся под размер входных данных, обеспечивая оптимальное распределение вычислений между потоками.

Общие сведения о программе

Программа выполняет умножение двух матриц, элементы которых представлены в виде комплексных чисел. Для оптимизации вычислений используется многопоточный подход, позволяющий распараллеливать вычисления для отдельных элементов результирующей матрицы.

Исходный код программы

include/complex.hpp

```
#pragma once
#include <iostream>
struct Complex {
    double real;
    double imag;

    Complex() : real(0), imag(0) {}
    Complex(double r, double i) : real(r), imag(i) {}

    Complex operator+(const Complex& other) const {
        return Complex(real + other.real, imag + other.imag);
    }

    Complex operator*(const Complex& other) const {
        return Complex(real * other.real - imag * other.imag,
                        real * other.imag + imag * other.real);
    }
};
```

include/multiply.hpp

```
#pragma once
#include "complex.hpp"
//заполняет матрицу случайными комплексными числами
void fillMatrix(Complex* matrix, int rows, int cols);
//выводит матрицу на экран
void printMatrix(const Complex* matrix, int rows, int cols);
//выполняет умножение определённых строк матрицы
void* multiplyRows(void* arg);
//управляет созданием потоков для умножения матриц
void multiplyMatrices(const Complex* A, const Complex* B, Complex* C, int A_rows, int A_cols, int B_cols, int numThreads);
```

src/multiply.cpp

```
#include <cstdlib> // генерация случ. чисел
#include <ctime>   // генерация случ. чисел
#include <pthread.h>
#include "multiply.hpp"

void fillMatrix(Complex* matrix, int rows, int cols) {
    srand(time(0));
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            matrix[i * cols + j] = Complex(rand() % 10, rand() % 10); // matrix - одномерный массив
        }
    }
}

void printMatrix(const Complex* matrix, int rows, int cols) {
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            std::cout << "(" << matrix[i * cols + j].real << ", " << matrix[i * cols + j].imag << ") ";
        }
        std::cout << std::endl;
    }
}

struct ThreadData {
    int startRow;          // диапазон строк матрицы для обработки потоком
    int endRow;
    const Complex* A;
    const Complex* B;
    Complex* C;            // вывод в матрицу C
    int A_cols;            // n столбцов в A
    int B_cols;            // n столбцов в B
};

void* multiplyRows(void* arg) {
    ThreadData* data = static_cast<ThreadData*>(arg); //привожу к типу threaddata
    указателя на arg поскольку в аргументе дал void
```

```

//static_cast<> — это оператор приведения типов
for (int i = data->startRow; i < data->endRow; ++i) { //иду по строкам
    for (int j = 0; j < data->B_cols; ++j) {          // иду по столбцам
        Complex sum;
        for (int k = 0; k < data->A_cols; ++k) { // суммирую произведения элементов А и Б
            sum = sum + data->A[i * data->A_cols + k] * data->B[k * data->B_cols + j];
        }
        data->C[i * data->B_cols + j] = sum; // запись в C
    }
}

return nullptr;
}

void multiplyMatrices(const Complex* A, const Complex* B, Complex* C, int A_rows, int
A_cols, int B_cols, int numThreads) {
    int rowsPerThread = A_rows / numThreads;          // кол-во строк матрицы А которые
обработываются одним потоком
    pthread_t* threads = new pthread_t[numThreads];    // пассив идентиф. потоков
    ThreadData* threadData = new ThreadData[numThreads]; // массив структур struct
ThreadData для каждого потока

    for (int i = 0; i < numThreads; ++i) {             // задаю поля для каждого потока
        threadData[i].A = A;
        threadData[i].B = B;
        threadData[i].C = C;
        threadData[i].A_cols = A_cols;
        threadData[i].B_cols = B_cols;
        threadData[i].startRow = i * rowsPerThread;
        threadData[i].endRow = (i + 1) * rowsPerThread;
        if (i == numThreads - 1) {                     //учитываю неравномерное деление строк
            threadData[i].endRow = A_rows;
        }
        pthread_create(&threads[i], nullptr, multiplyRows, &threadData[i]);
        //ук. на функцию; аргумент передаваемый в функцию
    }
    //приведенный к void
}

    for (int i = 0; i < numThreads; ++i) {
        pthread_join(threads[i], nullptr); // ожидаю завершение потоков
    }

    delete[] threads;
    delete[] threadData;
}

```

Пример вывода:

```
user@DESKTOP-
Q8E00MG:/mnt/c/Users/user/Desktop/Labs/os/lr2/build$
./run_tests
Running                               main()                               from
/mnt/c/Users/user/Desktop/Labs/os/lr2/googletest/google
test/src/gtest_main.cc
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from MatrixMultiplicationTest
[          RUN                               ]
MatrixMultiplicationTest.SingleThreadCorrectness
[                               OK           ]
MatrixMultiplicationTest.SingleThreadCorrectness (0 ms)
[          RUN                               ]
MatrixMultiplicationTest.MultiThreadCorrectness
[                               OK           ]
MatrixMultiplicationTest.MultiThreadCorrectness (0 ms)
[ RUN          ] MatrixMultiplicationTest.PerformanceTest
```

Вывод.

В ходе выполнения лабораторной работы были изучены принципы многопоточного программирования, включая создание и управление потоками с помощью функций `pthread_create` и `pthread_join`. Было выявлено, что чрезмерное увеличение количества потоков приводит к дополнительным накладным расходам, из-за чего программа может работать медленнее, чем при минимально необходимом количестве потоков.

Применение многопоточности в данной задаче позволило значительно ускорить вычисления по сравнению с последовательной реализацией, особенно при работе с большими матрицами.